

TO

Founder, 100monkeys.ai

FROM

A grateful operator · ar-fullsend/aegis-deploy

Thank you for AEGIS.

Your control plane ran a full intent-to-execution FSM on a GTX 1660 Ti, rootless Podman, Kali, and a local GGUF. This note is what we learned adapting it to a slow local SLM — and the improvements we made on top of your work.

Date 21 August 2026

Lab Kali GNU/Linux · rootless Podman · LM Studio llama.cpp CUDA

Repo github.com/ar-fullsend/aegis-deploy (fork of 100monkeys-ai stack)

1. With thanks — and a result

First: thank you. AEGIS is an unusually complete piece of systems work — orchestrator, Temporal FSM, SEAL envelopes, Keycloak, SeaweedFS, FUSE/NFS workspaces, FSAL isolation, and Zaru MCP as a tool proxy. Standing that up on a laptop is already a gift to operators. We spent 21 August 2026 putting it on Kali with rootless Podman and a local GGUF, and it **did the thing you designed it to do**.

`builtin-intent-to-execution` completed and printed `[0, 1, 1, 2, 3, 5, 8, 13, 21, 34]` — the first ten Fibonacci numbers for `n=10`. Your isolated Python container finished in **588 ms**. That number is the compliment: the sandbox, volume, and `ContainerRun` path are tight. The ~65 minute wall clock was almost entirely the local 27B SLM and a few layers we tuned so a slow box can still finish the FSM you wrote.

This note is not a complaint. It is a field report from someone who wanted your stack to shine on hardware nobody would pick for 27B, and who then patched the deploy repo so it could. Everything below is offered as **improvements on your work**, with gratitude, and as candidates to take upstream if they are useful.

2. Lab

Layer	What we actually ran
Host	Kali Rolling, 6 CPU threads, 15 GiB RAM, NVIDIA GeForce GTX 1660 Ti 6 GB (CUDA llama.cpp via LM Studio 0.4.21).
Runtime	Rootless Podman kube-play. Profile <code>development</code> : database, secrets, temporal, seal-gateway, iam, core, mcp, storage, observability.
LLM #1	<code>prism-ml/bonsai-27b</code> GGUF Q1_0, 4.73 GB, 56 GPU layers, ctx 8192, parallel 4, KV f16, vision mmproj loaded. Thinking on, then off.
LLM #2 (now live)	<code>qwen2.5-coder-7b-instruct</code> Q4_K_M from <code>lmstudio-community/Qwen2.5-Coder-7B-Instruct-GGUF</code> . Reloaded <code>--gpu max -c 4096 --parallel 1</code> . Smoke test: chat completion “pong” in 0.38 s , 2 completion tokens, 0 reasoning tokens.
Auth	Keycloak 24 realm <code>aegis-system</code> , client <code>aegis-runtime</code> , issuer <code>http://127.0.0.1:8180</code> . Local Admin UI is happiest at that URL; <code>auth.localhost + --proxy=edge</code> wants Caddy in front.

Table 1. Hardware and control-plane facts. Nothing here is hypothetical.

3. What the pipeline actually did

Workload: “Write a Python function `fib(n)` that returns the first `n` Fibonacci numbers,” language `python`, `inputs.n=10`, image `python:3.11-slim`. Workflow `builtin-intent-to-execution`.

Execution	Version	Outcome	Wall clock (approx.)
c2ac7aed... / WRITE 3372e7e9	workflow 1.0.0 (stock, in-flight at core boot)	WRITE completed; VALIDATE timed out at 300 s overall on first thinking generate. Never called fs.read.	WRITE ~4 min + VALIDATE 5 min fail
f6463680-0e2b-4c9f-bdb 8-e185bab6c5f1	1.0.2	LM Studio unreachable at pasta 169.254.1.2. WRITE/VALIDATE 502 then “Output is not valid JSON: EOF”.	~27 s fail
ac5367c2-bed4-48cf-b4f5 -4f2758381c26	1.0.2	WRITE+VALIDATE pass. Container exit 0 in 602 ms. Formatter 300 s × Temporal retries blew EXECUTE 15 min activity.	~22 min fail at EXECUTE
9a9a8908-818d-41c7-804 a-67e90bf2e94a	1.0.3 (formatter required:false)	completed. stdout was a single newline. Model wrote code that ran and printed nothing useful.	~20 min
19939581-953f-4c43-92a 5-97de0c2d958f	1.0.3, thinking off	completed. First EXECUTE exit 1, FSM looped, second EXECUTE stdout [0, 1, 1, 2, 3, 5, 8, 13, 21, 34], 588 ms, exit 0.	~65 min

Table 2. Intent-to-execution attempts on this node. The successful payload is yours.

The 65-minute success is a story about the model, not the architecture. Your three-agent FSM and Temporal formatter are doing exactly what they were designed to do — they simply assumed cloud-class generate latency. Once thinking was off, WRITE/VALIDATE were still multi-minute because Q1_0 27B on 6 GB is a packing trick, not a latency trick. The sandbox you built was never the bottleneck, and that is high praise.

4. Problems we solved on the local-SLM path

These are edge cases we hit while stretching AEGIS onto a 6 GB card and rootless netavark. We solved them in the deploy fork and would be glad to contribute the patches upstream. None of this diminishes the design — it is what operators do when they care about a system.

4.1 Host gateway on rootless netavark

The documented local-LLM path is `http://host.containers.internal:1234/v1`, which is the right abstraction. On this Kali + netavark bridge (`aegis-network 10.89.0.0/24`), Podman injects `169.254.1.2`. From `aegis-core` that address timed out on every port. The host LAN IPv4 (`192.168.1.204` today) worked. Bridge gateway `10.89.0.1:1234` did not — 1234 is a host process, not a published container port, so rootlessport never maps it.

What we did: rewrite `/etc/hosts` in the core pod at deploy/redeploy (`scripts/patch-host-gateway.sh`). Kubernetes `hostAliases` alone is not enough because pasta’s `169.254.1.2` line wins first-match. If local LLM is a SKU you want to support first-class, we would love to help land a tested host-gateway story for rootless netavark. Happy to send the script.

4.2 Naming the timeout layers

Your timeout model is actually quite careful — there are separate knobs for a single generate, an iteration, the whole agent, the Temporal state, the formatter activity, and the HTTP client. We learned that the hard way, and then we used that design as intended. A uniform “5 minutes everywhere” does not work until each layer is named:

Knob	Where	What it actually caps
<code>llm_timeout_seconds</code>	Agent spec.execution	One generate (HTTP to the provider).
<code>iteration_timeout</code>	Agent spec.execution	One supervisor iteration (generate + tools).
<code>security.resources.timeout</code>	Agent spec	Whole agent run. This became <code>overall_timeout_secs</code> . VALIDATE died here at 300 s on iteration 1.
<code>states.*.timeout</code>	Workflow FSM	Temporal activity StartToClose for that state.
<code>output_handler.timeout_seconds</code>	ContainerRun	Formatter activity. Stock $60\text{ s} \times 3\text{ retries}$ = “Activity task failed” at EXECUTE_CODE while the Python container already exited 0.
<code>llm_overall_timeout_secs</code>	NodeConfig	Orchestrator HTTP client. Must be $\geq$ the largest agent llm timeout or the client wins.

Table 3. Timeout fields in your manifests — we used each of them.

VALIDATE is a two-turn tool agent (mandatory `fs.read`, then JSON with `json_schema min_score 1.0`). That design is sound for a capable model. On a thinking 27B, setting `resources.timeout = llm_timeout` meant the second turn never started. We learned to set **overall** $\geq N \times$ **per-call** for iterative agents. A deploy-time check that refuses a manifest which cannot mathematically finish would be a lovely addition to the engine you already have.

4.3 Overlay versions above stock builtins

Core start helpfully re-registers stock v1.0.0 agents and the builtin workflow (`content drift detected — overwriting`). That is a reasonable way to keep builtins fresh. To keep our local-SLM timeouts, we versioned overlays at 1.0.1 / 1.0.2 / 1.0.3 / now **1.0.4** so semver stays above stock. That works for new runs. It does not rewrite in-flight executions started at boot (our first VALIDATE timeout was 1.0.0, started at 15:48:03, overlay applied 15:48:38).

A small upstream improvement: treat overlay/latest as an explicit node config, or skip force-redeploy when an operator overlay is already newer. We would be happy to sketch a lock file if that fits the model.

4.4 Formatter as best-effort on a slow SLM

The output formatter is a nice consumer-facing idea. On this box, `ContainerRunCompleted` in 602 ms, exit 0, then `aegis-output-formatter-agent` sat on a 27B generate until `overall_timeout_secs=300`. Temporal retried, and the parent `EXECUTE` state (15 m) ended with `Activity task failed`. The structured result already lives on `EXECUTE_CODE.stdout` via your `output_template` — which is elegant. We simply needed the pretty-print step not to own the critical path.

Version 1.0.4 uses `timeout_seconds: 15, required: false`, `EXECUTE` state 5 m, `max_state_visits: 1`. If the formatter cannot return in 15 s, we keep the raw stdout. For local SLM that feels like the right default; cloud SKUs can keep the formatter required. Your split of `required` already made that possible.

4.5 Auth scopes for the CLI we love

`aegis workflow run --follow` is a great operator experience. It 403'd without `workflow: logs`. Stock bootstrap listed `workflow: run/execute/deploy`. We added `workflow: logs`, `workflow: cancel`, `workflow: status`, and `execution: list` to `scripts/bootstrap-keycloak.sh`. Issuer is the browser-reachable URL (`127.0.0.1:8180`); JWKS from inside the mesh is `aegis-iam:8180`. For local MCP we used `BYPASS_AUTH=true` (still requires a Bearer string) while that split is sorted. Offering this as a bootstrap completeness patch, not a security argument.

4.6 VALIDATE on a small judge

The validator's contract is structural: `def solve(inputs)`, reads `INTENT_INPUTS`, prints JSON, `stdlib` only. That is a thoughtful gate. On a 7–27B local judge it is also a second generate plus a mandatory `fs.read`. On the successful run the judge wrote both “`INTENT_INPUTS` is empty/{ }” so this criterion is automatically satisfied” and “consumes inputs from environment” in the same object. The pipeline still passed because it emitted `valid: true`.

A deterministic AST/linter path for those six checks — with the LLM judge optional behind a flag, file bytes injected so you skip the tool-turn — would make your already-good gate cheaper on small models. We would use it immediately.

4.7 Metrics so Grafana agrees with /health

Grafana showed runtime, Keycloak, and OpenBao as down while `/health` was live. Scrapes hit loopback metrics on 9091 (the proxy is 9092); Keycloak needed `--metrics-enabled=true`; OpenBao needed an unauthenticated scrape listener. None of that is SLM-related. We fixed it locally so a first-time operator can trust the dashboards you already built. Those three flags belong in the default compose.

5. How we optimized the local model (no new GPU)

AEGIS is model-agnostic — that is one of its strengths. We started with Bonsai 27B Q1_0 because it *fits* 6 GB. It is not fast there. VRAM sat at 5880/6144 MiB. llama.cpp: 56 GPU layers, ctx 8192, parallel 4, KV cache f16, `--threads 2` on a 6-thread CPU, plus a BF16 mmproj for a coding task that never sees an image. NodeConfig had `context_window: 32768` and `max_output_tokens: 8192`, which is generous for cloud and expensive for a local generate. We tightened the aliases rather than fighting the orchestrator.

Change	Before	After	Why
Weights	Bonsai-27B Q1_0 (extreme quant, thinking)	Qwen2.5-Coder-7B-Instruct Q4_K_M	7B coder is the honest SKU for 6 GB. 27B Q1 still fits; we keep it on disk as a stretch option.
Thinking	On (first VALIDATE burned 300 s with no tool call)	Off	Reasoning tokens are not free on 6 GB. First success required this.
Load flags	ctx 8192, parallel 4, n-gpu-layers 25 (UI default on Qwen)	<code>lms load --gpu max -c 4096 --parallel 1 → n-gpu-layers 999999</code>	UI default only offloaded 25/28-ish Qwen layers. Max offload + parallel 1 is the 1660 Ti profile.
Aliases	All aliases → Bonsai, max_output 8192/4096	All aliases → <code>qwen2.5-coder-7b-instruct</code> , 2048 / judge 1024, ctx 4096, temp 0.3/0.1	Builtins request alias default. One wrong alias silently goes back to 27B.
Formatter	required, 300–600 s, Temporal ×3	required false, 15 s	Your output_template already has stdout. Formatter becomes best-effort locally.
FSM visits	WRITE/VALIDATE/EXECUTE visits 3; EXECUTION_FAILED loops to WRITE	visits 1	Retry loops are perfect for cloud. Local SLM fails once so we can inspect.

Table 4. Local-model changes we made so AEGIS could show off on a 1660 Ti.

Live check after reload, 21 August 2026 evening: LM Studio identifier `qwen2.5-coder-7b-instruct`, 4.36 GiB estimated, 5222 MiB VRAM used, ctx 4096, parallel 1, flash-attn on. Chat completions pong in 0.38 s. Core NodeConfig and overlay 1.0.4 are pointed at that id. Bonsai remains on disk as an optional stretch model.

Honest leftover knobs (LM Studio, not AEGIS): llama.cpp is still `--threads 2` because `lms load` has no threads flag; KV cache is still f16; the embedding model stays listed on `/v1/models`. The OpenAI proxy on 0.0.0.0:1234 is the correct target from pods once the host-gateway patch is applied.

6. What we added in the deploy fork

Fork: `github.com/ar-fullsend/aegis-deploy`, built on your stack. MCP wiring is already pushed; the Qwen, timeout, and host-gateway work is in the working tree and will land as Conventional Commits. These are operator improvements on top of AEGIS, offered upstream if you want them:

Area	Change
MCP	Pod <code>aegis-mcp</code> , image <code>ghcr.io/100monkeys-ai/zaru-mcp-server</code> , hostPort 8090, development+full profiles, Caddy <code>DOMAIN_MCP</code> , validate-stack health. <code>initialize</code> → <code>zaru-mcp-server 0.15.0-pre-alpha</code> ; <code>tools/list</code> returns orchestrator tools.
Overlays	<code>manifests/slow-slm/</code> now v1.0.4. Writer overall 15 m, validator 20 m / 10 m generate, formatter 10 m then 15 s <code>required=false</code> . <code>make overlays</code> after every core start.
Host gateway	<code>scripts/patch-host-gateway.sh</code> + detect LAN src from default route. Wired into <code>deploy.sh</code> and <code>make redeploy POD=core</code> .
Keycloak	hostname-url 127.0.0.1:8180, metrics enabled, extra workflow/execution scopes.
Observability	Runtime metrics-proxy :9092, OpenBao unauthenticated metrics on listener, Grafana jobs use <code>max(up{job=...})</code> where we touched dashboards.
FUSE	User unit <code>ExecStart=%h/.local/bin/aegis</code> , <code>StartLimitIntervalSec=0</code> . Kali is not Ubuntu; <code>setup.sh</code> had to be allowed to proceed.

Table 5. Deploy-fork additions we would be glad to contribute back.

7. Ideas we would love to see upstream

Your documentation is already ambitious and useful — we leaned on it all day. These are optional product enhancements, not a request for more prose. We would be honored if any of them fit the roadmap.

- A first-class local-SLM profile.** You already have profiles (minimal / development / full). A sibling that (a) maps host LLM via a tested gateway, (b) sets overall $\geq$ iterations $\times$ per-call, (c) best-efforts the formatter, (d) uses a cheap structural validator, (e) keeps operator overlays across core start would make AEGIS shine on the hardware people actually have.
- An optional deterministic VALIDATE.** Keep the LLM judge — it is a good idea — behind a flag, and inject file bytes so a small model does not pay a tool-turn. Today a 7B can claim `INTENT_INPUTS` is empty when the env is `{"n":10}`.
- Treat formatter as post-processing when ContainerRun exits 0.** Your `required` flag already supports this. Making best-effort the default on a local profile would spare operators a 15-minute `Activity task failed` after a 600 ms success.
- Bootstrap scopes that match the CLI.** Adding `workflow: logs` so `--follow` works out of the box. A short note on issuer vs JWKS for rootless pods would help everyone who copies the getting-started path.
- A recommended local model in getting started.** Qwen2.5-Coder-7B Q4_K_M is a kind default for 6–8 GB cards. 27B remains a great stretch goal when VRAM allows. AEGIS being model-agnostic is the point — a

suggested SKU just shortens the first afternoon.

6. **MCP health that names optional pieces.** Zaru as a SEAL proxy is the right shape — we wired `ghcr.io/100monkeys-ai/zaru-mcp-server` and it listed orchestrator tools immediately. Missing `ZARU_CLIENT_URL` only affects User Memory; surfacing that in the health payload would be a polish item, not a redesign.

8. Scope and humility

We have not yet load-tested Qwen through the full FSM; the 0.38 s pong is a provider smoke test, not a `WRITE+VALIDATE+EXECUTE` measurement. We have not enabled Gemini. We have not deployed edge TLS. We used `BYPASS_AUTH` on MCP as a local convenience while issuer and JWKS disagree — that is not a production posture. OpenBao root tokens live in `generated/`, gitignored, as your layout intended.

What we **are** glad to report: your control plane executed user-generated Python in an isolated container, with a workspace volume, a Temporal FSM, and a correct numeric result, on a GTX 1660 Ti. That is the system you built. We are grateful for it, and we hope the local-SLM work helps the next operator get there faster.

Appendix — knobs live on this node

LM Studio load: `lms load qwen2.5-coder-7b-instruct --gpu max -c 4096 --parallel 1 -y`. AEGIS apply: `make overlays` after every `podman restart aegis-core-aegis-runtime`. Host gateway: `make redeploy POD=core` already calls `scripts/patch-host-gateway.sh`. Token: `client_credentials aegis-runtime` against realm `aegis-system`. Successful fib execution id: `19939581-953f-4c43-92a5-97de0c2d958f`.

UIs (all 127.0.0.1): runtime 8088, Keycloak 8180, Temporal 8233, SEAL 8089, MCP 8090, OpenBao 8200, Grafana 3300, Prometheus 9090, Jaeger 16686, Seaweed filer 8888 / master 9333, LM Studio 1234.

The complete unified diff is the next appendix — every overlay, config, script, and doc change from today, so you can read or apply it without cloning our fork.

Appendix — unified diff of the deploy fork

Offered with thanks, as a patch you can read or apply. This is `git diff HEAD of ar-fullsend/aegis-deploy` against the tree we started from today, plus three new scripts. Excluded: this PDF, its generator, and an unrelated `pc-tuning/` folder. Added lines are prefixed with + (dark green). Removed lines are prefixed with – (dark red). Color is not the only cue.

Path	Added	Removed
.env.example	4	0
AGENTS.md	4	2
Makefile	11	1
README.md	4	2
docs/LOCAL-OPS.md	21	18
manifests/slow-slm/README.md	22	13
manifests/slow-slm/aegis-bash-executor-agent.yaml	2	2
manifests/slow-slm/aegis-code-validator-agent.yaml	9	5
manifests/slow-slm/aegis-javascript-executor-agent.yaml	2	2
manifests/slow-slm/aegis-output-formatter-agent.yaml	6	6
manifests/slow-slm/aegis-python-executor-agent.yaml	2	2
manifests/slow-slm/builtin-intent-to-execution.yaml	7	7
podman/pods/core/aegis-config.yaml	26	24
podman/pods/core/pod-core.yaml	7	0
scripts/apply-slow-slm-overlays.sh	76	0
scripts/bootstrap-keycloak.sh	2	1
scripts/deploy.sh	20	0
scripts/lib/host-lan-ip.sh	10	0
scripts/patch-host-gateway.sh	45	0

Table 6. 19 paths, 280 insertions, 85 deletions. Full unified diff follows.

.env.example

```
diff --git a/.env.example b/.env.example
index 2329f9b..3728e24 100644
--- a/.env.example
+++ b/.env.example
@@ -64,6 +64,10 @@ CORTEX_GRPC_URL=http://aegis-cortex:50052
# Create at: https://aistudio.google.com/app/apikey
ZARU_LLM_API_KEY=

+# Host IPv4 used inside pods for host.containers.internal (LM Studio + FUSE).
+# Leave empty to auto-detect the default-route src address at deploy time.
+# AEGIS_HOST_LAN_IP=
+
# --- IAM (Keycloak) ---
KEYCLOAK_ADMIN_PASSWORD=changeme
# Client secret for aegis-temporal-worker in the aegis-system realm.
```

AGENTS.md

```
diff --git a/AGENTS.md b/AGENTS.md
index a8a3831..020c74b 100644
--- a/AGENTS.md
+++ b/AGENTS.md
@@ -95,7 +95,7 @@ The observability pipeline: services export OTLP to **otelcol** at `aegis-observ
1. **Read `docs/LOCAL-OPS.md` first** – WSL2 / LLM / JWT / Temporal vs agent execute.
2. **WSL2:** Windows browser must use WSL eth0 IP, never `127.0.0.1`.
-3. **LLM:** `host.containers.internal:1234`; LM Studio bind `0.0.0.0`; literal model ids
(`prism-ml/bonsai-27b`); Gemini off unless keyed.
+3. **LLM:** `host.containers.internal:1234`; LM Studio bind `0.0.0.0`; literal model id
`qwen2.5-coder-7b-instruct` (Q4_K_M); Gemini off unless keyed. On Kali/netavark, `make deploy` / `make
redploy POD=core` must patch `host.containers.internal` to the host LAN IP
(`scripts/patch-host-gateway.sh`) – pasta's `169.254.1.2` times out.
4. **Auth:** `aegis-runtime` / **`placeholder`**, issuer `http://127.0.0.1:8180`. Not
`AEGIS_API_TOKEN`. Not `auth.localhost`.
5. **`aegis agent run` is not a Temporal workflow.** Temporal UI only shows `aegis workflow run`.
6. **Profiles** live in `profiles/*.conf` – trust the conf files.
@@ -124,7 +124,8 @@ PROFILE=minimal make deploy
PROFILE=full make deploy

# Redeploy a single pod after config change
-make redeploy POD=core
+make redeploy POD=core # also patches host.containers.internal and re-applies overlays
+make overlays # re-apply slow-SLM timeouts without a core restart
make redeploy POD=observability

# Tail logs
@@ -155,6 +156,7 @@ make status
5. Restarts the systemd FUSE daemon to pick up the new binary
6. Iterates `$PODS` in order, running `envsubst < pod-*.yaml | podman play kube --network aegis-network
--replace -`
7. After deploying `pod-secrets`, auto-runs `bootstrap-openbao.sh` and re-sources `.env`
+8. After all pods, applies `manifests/slow-slm/` overlays (`scripts/apply-slow-slm-overlays.sh`) so
local SLM timeouts stay latest

## FUSE Daemon (ADR-107)
```

Makefile

```
diff --git a/Makefile b/Makefile
index 796af1c..a3bac90 100644
--- a/Makefile
+++ b/Makefile
@@ -1,4 +1,4 @@
-.PHONY: setup deploy teardown status networks validate registry-login install-cli aegis shell
bootstrap-secrets bootstrap-keycloak generate-keys redeploy logs clean
+.PHONY: setup deploy teardown status networks validate registry-login install-cli aegis shell
bootstrap-secrets bootstrap-keycloak generate-keys redeploy logs clean overlays

PROFILE ?= development
SCRIPTS := ./scripts
@@ -61,6 +61,10 @@ bootstrap-keycloak:
generate-keys:
@bash $(SCRIPTS)/generate-seal-keys.sh

+# ---- Slow-SLM overlays (re-apply after core restart) ----
+overlays:
+ @bash $(SCRIPTS)/apply-slow-slm-overlays.sh
+
# ---- Redeploy single pod ----
redeploy: registry-login
@if [ -z "$(POD)" ]; then echo "Usage: make redeploy POD=core"; exit 1; fi
@@ -68,7 +72,13 @@ redeploy: registry-login
@set -a && . ./env && set +a && \
POD_FILE=$(find $(PODS_DIR)/$(POD) -name "pod-*.yaml" -type f | head -1) && \
if [ -z "$$POD_FILE" ]; then echo "ERROR: No pod YAML found in $(PODS_DIR)/$(POD)"; exit 1; fi && \
+ AEGIS_HOST_LAN_IP=${AEGIS_HOST_LAN_IP:-$(bash $(SCRIPTS)/lib/host-lan-ip.sh)} && \
+ export AEGIS_HOST_LAN_IP && \
envsubst < "$$POD_FILE" | podman play kube --network aegis-network --replace - && \
+ if [ "$(POD)" = "core" ]; then \
+ bash $(SCRIPTS)/patch-host-gateway.sh && \
+ bash $(SCRIPTS)/apply-slow-slm-overlays.sh; \
+ fi && \
echo "==> Pod $(POD) redeployed"

# ---- Helpers ----
```

README.md

```
diff --git a/README.md b/README.md
index fc78009..6414b49 100644
--- a/README.md
+++ b/README.md
@@ -127,8 +127,8 @@ See [docs/LOCAL-OPS.md](docs/LOCAL-OPS.md) for the full UI port list, LLM config
- Host server: `http://127.0.0.1:1234` — bind **`0.0.0.0`** (`lms server start --bind 0.0.0.0`).
`127.0.0.1` is unreachable from pods.
- From pods: `http://host.containers.internal:1234/v1`
-- Model: `prism-ml/bonsai-27b` (confirm with `curl -s http://127.0.0.1:1234/v1/models`)
-- Aliases `default`, `slm`, `smart`, `judge`, `slm-judge` all point at Bonsai (`max_output_tokens`
8192; thinking models spend tokens on `reasoning_content` first)
+- Model: `qwen2.5-coder-7b-instruct` (Qwen2.5-Coder-7B-Instruct Q4_K_M; confirm with `curl -s
http://127.0.0.1:1234/v1/models`)
+- Aliases `default`, `slm`, `smart`, `judge`, `slm-judge` all point at Qwen Coder 7B
(`max_output_tokens` 2048, 1024 for judge; ctx 4096)
- Gemini stays `enabled: false` unless `ZARU_LLM_API_KEY` is set

After config changes: `podman restart aegis-core-aegis-runtime`.
@@ -146,6 +146,7 @@ After config changes: `podman restart aegis-core-aegis-runtime`.
| `make bootstrap-secrets` | Initialize OpenBao and populate AppRole credentials |
| `make bootstrap-keycloak` | Configure Keycloak realm, clients, and roles |
| `make generate-keys` | Generate SEAL RSA signing key pair |
```

```
+| `make overlays` | Re-apply `manifests/slow-slm/` agent/workflow versions (after core restart) |
| `make redeploy POD=<name>` | Tear down and redeploy a single pod |
| `make logs POD=<name>` | Tail logs for a specific pod |
| `make clean` | Full teardown + prune volumes and networks |
@@ -161,6 +162,7 @@ Copy `.env.example` to `.env` and fill in the required values. Key variables:
| `GHC_TOKEN` | Yes | GitHub PAT with `read:packages` scope |
| `POSTGRES_PASSWORD` | Yes | PostgreSQL password |
| `ZARU_LLM_API_KEY` | No | Gemini key; unused while Gemini is disabled |
+| `AEGIS_HOST_LAN_IP` | No | Host IPv4 for `host.containers.internal` (auto-detected if empty) |
| `KEYCLOAK_ADMIN_PASSWORD` | Recommended | Keycloak admin password (default: `changeme`) |
| `GRAFANA_ADMIN_PASSWORD` | Recommended | Grafana admin password (default: `changeme`) |
| `CLOUDFLARE_API_TOKEN` | Edge only | Required for Caddy TLS via DNS challenge |
```

docs/LOCAL-OPS.md

```
diff --git a/docs/LOCAL-OPS.md b/docs/LOCAL-OPS.md
index 8ea2ffc..c631bae 100644
--- a/docs/LOCAL-OPS.md
+++ b/docs/LOCAL-OPS.md
@@ -49,10 +49,12 @@ File: `podman/pods/core/aegis-config.yaml`
- **Strategy:** `prefer-local`, `default_provider: lmstudio`
- **LM Studio (host):** `http://127.0.0.1:1234` – server is already ON.
- **From pods:** `http://host.containers.internal:1234/v1` (container `127.0.0.1` is not the host). LM
Studio must bind `0.0.0.0` (`lms server start --bind 0.0.0.0`), not `127.0.0.1`.
-- **Model:** [prism-ml/bonsai-27b](https://lmstudio.ai/models/prism-ml/bonsai-27b) (`Bonsai-27B-Q1_0`
GGUF). Load it in LM Studio, then confirm the API id:
+- **Kali + netavark:** Podman injects `host.containers.internal` → `169.254.1.2`, which **times out**
from `aegis-network`. `scripts/patch-host-gateway.sh` rewrites it to the host LAN IPv4 (default-route
`src`). `make deploy` / `make redeploy POD=core` run that patch. Verify: `podman exec
aegis-core-aegis-runtime curl -sS http://host.containers.internal:1234/v1/models`.
+- **Model:** [qwen2.5-coder-7b-instruct](https://huggingface.co/lmstudio-community/Qwen2.5-Coder-7B
-Instruct-GGUF) (`Q4_K_M`). Confirm id:
`curl -s http://127.0.0.1:1234/v1/models`
- If the `id` differs from `prism-ml/bonsai-27b`, update every `model:` field in `aegis-config.yaml`.
-- **Aliases on lmstudio:** `default`, `slm`, `smart`, `judge`, `slm-judge` – all point at Bonsai.
Built-ins request `default`.
+ If the `id` differs, update every `model:` field in `aegis-config.yaml`.
+- **Load (GTX 1660 Ti 6GB):** `lms load qwen2.5-coder-7b-instruct --gpu max -c 4096 --parallel 1 -y`.
Thinking **off**. Unload Bonsai first (`lms unload --all`).
+- **Aliases on lmstudio:** `default`, `slm`, `smart`, `judge`, `slm-judge` – all point at Qwen Coder
7B. Built-ins request `default`. `max_output_tokens` is 2048 (1024 for judge), not 8192.
- **Gemini:** `enabled: false` unless `ZARU_LLM_API_KEY` is set.
- After edits: `podman restart aegis-core-aegis-runtime` (config is hostPath-mounted).
@@ -73,7 +75,7 @@ TOKEN=$(curl -sS -X POST 'http://127.0.0.1:8180/realms/aegis-system/protocol/ope
| python3 -c 'import sys,json;print(json.load(sys.stdin)["access_token"])')
...

-Requires `make bootstrap-keycloak` once (idempotent). Re-run after changing scopes. The client needs
`agent:list`, `agent:execute`, `agent:deploy`, `execution:read`, `workflow:run`, and related scopes
assigned as default client scopes.
+Requires `make bootstrap-keycloak` once (idempotent). Re-run after changing scopes. The client needs
`agent:list`, `agent:execute`, `agent:deploy`, `execution:read`, `workflow:run`, `workflow:logs`
(required for `aegis workflow run --follow`), and related scopes assigned as default client scopes.

## Agent execute vs Temporal workflows

@@ -113,7 +115,7 @@ Confirm LLM path:
```bash
podman logs --since 5m aegis-core-aegis-runtime 2>&1 | grep -E 'LLM inference|LLM HTTP'
-# expect: model=prism-ml/bonsai-27b endpoint=http://host.containers.internal:1234/v1/chat/completions
+# expect: model=qwen2.5-coder-7b-instruct
endpoint=http://host.containers.internal:1234/v1/chat/completions
...

```

```

Register a workflow record (YAML)
@@ -216,24 +218,25 @@ Local tools on the MCP server (not forwarded): `zaru.init`, `zaru.mode`. Everyth
Docs: https://docs.100monkeys.ai/docs/zaru/mcp-client-setup

-## Execution timeouts (5 minutes)
+## Execution timeouts (slow SLM)

-Local SLM overlays in `manifests/slow-slm/` set 5 minutes on every execution layer (right field
for each):
+Local SLM overlays in `manifests/slow-slm/`: writers v1.0.2, formatter v1.0.3, workflow
v1.0.4. A 5-minute overall agent budget is not enough for VALIDATE_CODE: that agent must
`fs.read` then emit JSON, and a thinking 27B can spend the entire 300s on the first generate
(`Execution timed out after 300 seconds` at `final_state: VALIDATE_CODE`).

-| Layer | Field | Value |
-|---|---|---|
-| LLM generate | `llm_timeout_seconds` / `llm_overall_timeout_secs` | `300` |
-| Agent iteration | `iteration_timeout` | `5m` |
-| Agent resource wall clock | `security.resources.timeout` | `5m` |
-| Workflow state (WRITE/VALIDATE/EXECUTE) | `states.*.timeout` | `5m` |
-| Isolated code run | `EXECUTE_CODE.resources.timeout` | `5m` |
-| Formatter Temporal activity | `output_handler.timeout_seconds` | `300` |
+| Layer | Field | Writer | Validator |
+|---|---|---|---|
+| LLM generate | `llm_timeout_seconds` | `300` | `600` |
+| Orchestrator LLM HTTP client | `llm_overall_timeout_secs` | `180` | `180` |
+| Agent iteration | `iteration_timeout` | `5m` | `10m` |
+| Agent run wall clock | `security.resources.timeout` | `15m` | `20m` |
+| Workflow state | `states.*.timeout` | WRITE/VALIDATE/EXECUTE | `5m` | same |
+| Isolated code run | `EXECUTE_CODE.resources.timeout` | `5m` | n/a |
+| Formatter Temporal activity | `output_handler.timeout_seconds` | `15` (required: false) | n/a |

-Overlays are v1.0.1 so stock v1.0.0 builtins re-applied on core start stay underneath. Latest
= 5 minute timeouts.
+`make deploy` and `make redeploy POD=core` apply overlays after core is healthy. `make overlays` is
the standalone re-apply. Core start re-registers stock v1.0.0 builtins; overlay versions stay
latest.

-A Temporal `Activity task failed` at `EXECUTE_CODE` with 3 iterations is usually the formatter
activity, not the Python container.
+A Temporal `Activity task failed` at `EXECUTE_CODE` with 3 iterations is usually the formatter
activity, not the Python container. Overlay 1.0.4 makes the formatter optional so a slow format
cannot fail a successful container run.

-Thinking 27B models may still exceed 5 minutes – disable Enable Thinking in LM Studio if needed.
+Disable Enable Thinking in LM Studio if a single generate still stalls past 10 minutes.

Known non-blockers

```

## manifests/slow-slm/README.md

```

diff --git a/manifests/slow-slm/README.md b/manifests/slow-slm/README.md
index e4c8db8..a64c9b9 100644
--- a/manifests/slow-slm/README.md
+++ b/manifests/slow-slm/README.md
@@ -1,19 +1,28 @@
-# Local SLM timeouts (5 minutes)
+# Local SLM timeouts

-All execution timeouts in this directory are 5 minutes, using the field that matches each
layer:
+Stock builtins use one 5-minute budget for the entire agent run. That is not enough for a thinking
27B: VALIDATE_CODE must `fs.read` then emit JSON (two LLM generates). A single generate can consume the
whole 300s wall clock (`overall_timeout_secs=300`) and the workflow fails at VALIDATE with `Execution
timed out after 300 seconds`.

-| Context | Field | Value |

```

```

-|---|---|---|
-| Single LLM HTTP generate | `spec.execution.llm_timeout_seconds` | `300` |
-| One agent iteration | `spec.execution.iteration_timeout` | `5m` |
-| Agent container / FSAL wall clock | `spec.security.resources.timeout` | `5m` |
-| Orchestrator-wide LLM generate | `llm_selection.llm_overall_timeout_secs` in `aegis-config.yaml` | `300` |
-| Workflow Agent/ContainerRun **state** | `spec.states.*.timeout` | `5m` |
-| Isolated code container | `EXECUTE_CODE.resources.timeout` | `5m` |
-| Temporal output-handler activity | `output_handler.timeout_seconds` | `300` |
+Overlays split the knobs:
+
+| Context | Field | Writer / formatter | Validator |
+|---|---|---|---|
+| Single LLM HTTP generate | `spec.execution.llm_timeout_seconds` | `300` | `600` |
+| One agent iteration | `spec.execution.iteration_timeout` | `5m` | `10m` |
+| Agent run wall clock (`overall_timeout_secs`) | `spec.security.resources.timeout` | `15m` | `20m` |
+| Orchestrator LLM HTTP client | `llm_selection.llm_overall_timeout_secs` in `aegis-config.yaml` | `180` | `180` |
+| Workflow Agent/ContainerRun **state** | `spec.states.*.timeout` | WRITE/VALIDATE/EXECUTE `5m` | same |
+| Isolated code container | `EXECUTE_CODE.resources.timeout` | `5m` | n/a |
+| Formatter agent wall clock | `aegis-output-formatter-agent` `resources.timeout` | `10m` | n/a |
+| Temporal output-handler activity | `output_handler.timeout_seconds` | `15` (`required: false`) | n/a |
|
Not changed: OTLP exporter `5s`, health probes, Prometheus scrapes, TCP checks in `validate-stack.sh`.
+```bash
+make overlays
+```
+
+Equivalent:
+
+```bash
TOKEN=$(curl -sS -X POST 'http://127.0.0.1:8180/realms/aegis-system/protocol/openid-connect/token' \
-H 'Content-Type: application/x-www-form-urlencoded' \
@@ -28,6 +37,6 @@ aegis --host 127.0.0.1 --port 8088 workflow deploy --force --scope global \
manifests/slow-slm/builtin-intent-to-execution.yaml
```

-Overlays use **version 1.0.1** so a core restart that re-deploys stock **1.0.0** builtins does not become latest. `aegis workflow run` / agent execute use latest.
+Overlay versions stay above stock **1.0.0** so a core restart that re-deploys builtins does not become latest: writers **1.0.2**, formatter **1.0.3**, workflow **1.0.4**. `aegis workflow run` / `aegis.execute.intent` use latest. `make deploy` and `make redeploy POD=core` apply these overlays after core is healthy; `make overlays` re-applies them on demand.

-A thinking 27B may still exceed 5 minutes; disable **Enable Thinking** in LM Studio if generate calls stall.
+Disable **Enable Thinking** in LM Studio if a single generate still stalls past 10 minutes.

```

manifests/slow-slm/aegis-bash-executor-agent.yaml

```

diff --git a/manifests/slow-slm/aegis-bash-executor-agent.yaml
b/manifests/slow-slm/aegis-bash-executor-agent.yaml
index 281ccb8..7e98fe6 100644
--- a/manifests/slow-slm/aegis-bash-executor-agent.yaml
+++ b/manifests/slow-slm/aegis-bash-executor-agent.yaml
@@ -2,7 +2,7 @@ apiVersion: 100monkeys.ai/v1
kind: Agent
metadata:
name: aegis-bash-executor-agent
- version: 1.0.1
+ version: 1.0.2

```

description: Writes a bash script that fulfills a given intent, self-tests it via cmd.run against the actual INTENT_INPUTS plus a derived case, and saves it to a workspace file. Used by the intent-to-execution pipeline as the WRITE_CODE step.

labels:

builtin: 'true'

@@ -115,7 +115,7 @@ spec:

cpu: 500

memory: 256Mi

disk: 1Gi

- timeout: 5m

+ timeout: 15m

tools:

- fs.write

- cmd.run

manifests/slow-slm/aegis-code-validator-agent.yaml

diff --git a/manifests/slow-slm/aegis-code-validator-agent.yaml

b/manifests/slow-slm/aegis-code-validator-agent.yaml

index 1c71e1a..448ac2e 100644

--- a/manifests/slow-slm/aegis-code-validator-agent.yaml

+++ b/manifests/slow-slm/aegis-code-validator-agent.yaml

@@ -2,7 +2,7 @@ apiVersion: 100monkeys.ai/v1

kind: Agent

metadata:

name: aegis-code-validator-agent

- version: 1.0.1

+ version: 1.0.2

description: Validates generated code is a reusable function that reads inputs from the INTENT_INPUTS environment variable rather than hardcoding literals. Returns structured JSON with a valid field used by workflow transition conditions to gate execution.

labels:

role: judge

@@ -14,7 +14,7 @@ spec:

version: '3.11'

image_pull_policy: IfNotPresent

isolation: inherit

- model: smart

+ model: judge

temperature: 0.1

task:

instruction: |

@@ -22,6 +22,10 @@ spec:

from the workspace and verify it meets the structural requirements for the intent-to-execution pipeline.

+ SPEED: Do not narrate, plan, or restate the criteria. First message
+ MUST be an fs.read tool call. Second message MUST be the JSON object
+ only (no markdown fences, no prose).

+

YOUR FIRST ACTION MUST be to call fs.read with the ABSOLUTE path shown in the "File:" field in the input below. The path will always start with /workspace/ (e.g. /workspace/solution.py). You MUST use

@@ -99,8 +103,8 @@ spec:

execution:

mode: iterative

max_retries: 1

- llm_timeout_seconds: 300

- iteration_timeout: 5m

+ llm_timeout_seconds: 600

+ iteration_timeout: 10m

validation:

- type: json_schema

```

schema:
@@ -137,7 +141,7 @@ spec:
cpu: 500
memory: 256Mi
disk: 1Gi
- timeout: 5m
+ timeout: 20m
tools:
- fs.read
security_context: aegis-system-agent-runtime

```

manifests/slow-slm/aegis-javascript-executor-agent.yaml

```

diff --git a/manifests/slow-slm/aegis-javascript-executor-agent.yaml
b/manifests/slow-slm/aegis-javascript-executor-agent.yaml
index cc282fb..3cf1e3e 100644
--- a/manifests/slow-slm/aegis-javascript-executor-agent.yaml
+++ b/manifests/slow-slm/aegis-javascript-executor-agent.yaml
@@ -2,7 +2,7 @@ apiVersion: 100monkeys.ai/v1
kind: Agent
metadata:
name: aegis-javascript-executor-agent
- version: 1.0.1
+ version: 1.0.2
description: Writes idiomatic JavaScript (Node.js) code that fulfills a given intent, self-tests it via
cmd.run against the actual INTENT_INPUTS plus a derived case, and saves it to a workspace file. Used by
the intent-to-execution pipeline as the WRITE_CODE step.
labels:
builtin: 'true'
@@ -111,7 +111,7 @@ spec:
cpu: 500
memory: 256Mi
disk: 1Gi
- timeout: 5m
+ timeout: 15m
tools:
- fs.write
- cmd.run

```

manifests/slow-slm/aegis-output-formatter-agent.yaml

```

diff --git a/manifests/slow-slm/aegis-output-formatter-agent.yaml
b/manifests/slow-slm/aegis-output-formatter-agent.yaml
index b880e63..edba950 100644
--- a/manifests/slow-slm/aegis-output-formatter-agent.yaml
+++ b/manifests/slow-slm/aegis-output-formatter-agent.yaml
@@ -2,7 +2,7 @@ apiVersion: 100monkeys.ai/v1
kind: Agent
metadata:
name: aegis-output-formatter-agent
- version: 1.0.1
+ version: 1.0.3
description: General-purpose builtin output formatter. Receives raw execution output and contextual
instructions. Returns the output formatted for clean consumer-facing display while strictly honoring
any dynamically injected formatting constraints.
labels:
category: platform
@@ -14,10 +14,10 @@ spec:
version: '3.11'
image_pull_policy: IfNotPresent
isolation: inherit
- model: smart

```



```

+ model: judge
temperature: 0.1
task:
- instruction: "You are a deterministic display formatter. Your only job is to take raw output \nand
return it in the cleanest, most readable form for a human consumer.\n\nRules:\n1. CONTENT PRESERVATION:
Do not alter the factual meaning, numbers, or logic \n of the raw output. You are formatting, not
recalculating.\n2. OVERRIDE HIERARCHY: If the provided input contains specific formatting \n
instructions (e.g., \"one sentence or less\", \"do not include code\"), you \n MUST prioritize those
injected instructions over these general rules.\n3. STRUCTURED DATA: If the output is purely JSON or
YAML and no contrary \n instructions exist, preserve it exactly and wrap it in a fenced code block.\n4.
CLEAN TEXT: If the output is plain text, format it as clean markdown. \n Remove trailing newlines,
repr() wrappers, debug print prefixes (e.g., \n 'Fetched content:'), stack-frame noise, and ANSI escape
codes.\n5. NO CHATTER: Do not add commentary, pleasantries, or explanations. Do not \n say \"Here is
the result:\". Return ONLY the formatted answer.\n6. EMPTY STATE: If the raw output is completely empty
but the execution was \n successful, return exactly: \"Task executed successfully (No output
returned).\n\n\"
+ instruction: "You are a deterministic display formatter. SPEED: return the formatted output
immediately with no planning or preamble.\n\nYour only job is to take raw output \nand return it in the
cleanest, most readable form for a human consumer.\n\nRules:\n1. CONTENT PRESERVATION: Do not alter the
factual meaning, numbers, or logic \n of the raw output. You are formatting, not recalculating.\n2.
OVERRIDE HIERARCHY: If the provided input contains specific formatting \n instructions (e.g., \"one
sentence or less\", \"do not include code\"), you \n MUST prioritize those injected instructions over
these general rules.\n3. STRUCTURED DATA: If the output is purely JSON or YAML and no contrary \n
instructions exist, preserve it exactly and wrap it in a fenced code block.\n4. CLEAN TEXT: If the
output is plain text, format it as clean markdown. \n Remove trailing newlines, repr() wrappers, debug
print prefixes (e.g., \n 'Fetched content:'), stack-frame noise, and ANSI escape codes.\n5. NO CHATTER:
Do not add commentary, pleasantries, or explanations. Do not \n say \"Here is the result:\". Return
ONLY the formatted answer.\n6. EMPTY STATE: If the raw output is completely empty but the execution was
\n successful, return exactly: \"Task executed successfully (No output returned).\n\n\"
prompt_template: |
{{instruction}}

@@ -26,8 +26,8 @@ spec:
execution:
mode: iterative
max_retries: 1
- llm_timeout_seconds: 300
- iteration_timeout: 5m
+ llm_timeout_seconds: 600
+ iteration_timeout: 10m
security:
network:
mode: none
@@ -39,5 +39,5 @@ spec:
cpu: 250
memory: 128Mi
disk: 1Gi
- timeout: 5m
+ timeout: 10m
security_context: aegis-system-agent-runtime

```

manifests/slow-slm/aegis-python-executor-agent.yaml

```

diff --git a/manifests/slow-slm/aegis-python-executor-agent.yaml
b/manifests/slow-slm/aegis-python-executor-agent.yaml
index 7858d80..b37d591 100644
--- a/manifests/slow-slm/aegis-python-executor-agent.yaml
+++ b/manifests/slow-slm/aegis-python-executor-agent.yaml
@@ -2,7 +2,7 @@ apiVersion: 100monkeys.ai/v1
kind: Agent
metadata:
name: aegis-python-executor-agent
- version: 1.0.1

```

```
+ version: 1.0.2
description: Writes idiomatic Python 3 code that fulfills a given intent, self-tests it via cmd.run
against the actual INTENT_INPUTS plus a derived case, and saves it to a workspace file. Used by the
intent-to-execution pipeline as the WRITE_CODE step.
labels:
role: worker
@@ -107,7 +107,7 @@ spec:
cpu: 500
memory: 256Mi
disk: 1Gi
- timeout: 5m
+ timeout: 15m
tools:
- fs.write
- cmd.run
```

manifests/slow-slm/builtin-intent-to-execution.yaml

```
diff --git a/manifests/slow-slm/builtin-intent-to-execution.yaml
b/manifests/slow-slm/builtin-intent-to-execution.yaml
index 4f859ca..29dd242 100644
--- a/manifests/slow-slm/builtin-intent-to-execution.yaml
+++ b/manifests/slow-slm/builtin-intent-to-execution.yaml
@@ -3,7 +3,7 @@ apiVersion: 100monkeys.ai/v1
kind: Workflow
metadata:
name: builtin-intent-to-execution
- version: 1.0.1
+ version: 1.0.4
description: Seamless intent-to-execution pipeline. Accepts a natural-language intent and optional
typed inputs, dispatches to a builtin language executor agent to write code to an ephemeral workspace
volume, executes it in an isolated container, and returns a formatted result via output_handler.
Invoked via the aegis.execute.intent MCP tool from Zaru Execute mode.
labels:
type: builtin-execute
@@ -97,14 +97,14 @@ spec:
Preserve all data from the output. If the output is structured
(JSON, table, list), keep it intact. Clean up formatting artifacts
but do not summarize or condense.
- timeout_seconds: 300
- required: true
+ timeout_seconds: 15
+ required: false
transitions:
- condition: exit_code_non_zero
target: EXECUTION_FAILED
feedback: null
timeout: 5m
- max_state_visits: 3
+ max_state_visits: 1
VALIDATE_CODE:
kind: Agent
agent: aegis-code-validator-agent
@@ -131,7 +131,7 @@ spec:
target: WRITE_FAILED
feedback: null
timeout: 5m
- max_state_visits: 3
+ max_state_visits: 1
WRITE_CODE:
kind: Agent
agent: aegis-{{input.language}}-executor-agent
```

```

@@ -173,7 +173,7 @@ spec:
target: WRITE_FAILED
feedback: null
timeout: 5m
- max_state_visits: 3
+ max_state_visits: 1
WRITE_FAILED:
kind: System
command: echo 'Pipeline failed at WRITE_CODE'
@@ -192,7 +192,7 @@ spec:
target: WRITE_CODE
feedback: null
timeout: null
- max_state_visits: 2
+ max_state_visits: 1
storage:
workspace:
storage_class: ephemeral

```

podman/pods/core/aegis-config.yaml

```

diff --git a/podman/pods/core/aegis-config.yaml b/podman/pods/core/aegis-config.yaml
index 69bc8b6..0aaa099 100644
--- a/podman/pods/core/aegis-config.yaml
+++ b/podman/pods/core/aegis-config.yaml
@@ -16,8 +16,9 @@ spec:
  llm_providers:
    # Host LM Studio at 127.0.0.1:1234. From inside the pod that is
    # host.containers.internal (not 127.0.0.1 – that is the container loopback).
    - # Model: prism-ml/bonsai-27b (GGUF Bonsai-27B-Q1_0). Confirm id with
    - # curl http://127.0.0.1:1234/v1/models after it is loaded.
    + # Model: qwen2.5-coder-7b-instruct (Qwen2.5-Coder-7B-Instruct Q4_K_M).
    + # Confirm id: curl http://127.0.0.1:1234/v1/models after load.
    + # GTX 1660 Ti 6GB: GPU offload max, ctx 4096, parallel 1. Thinking off.
    - name: lmstudio
    type: openai
    endpoint: "http://host.containers.internal:1234/v1"
@@ -27,39 +28,39 @@ spec:
    # Built-ins often request alias "default" – must exist on local provider.
    # Use literal model id (env: expansion is unreliable for model field).
    - alias: "default"
    - model: "prism-ml/bonsai-27b"
    + model: "qwen2.5-coder-7b-instruct"
    capabilities: ["code", "reasoning"]
    - context_window: 32768
    - max_output_tokens: 8192
    - temperature: 0.7
    + context_window: 4096
    + max_output_tokens: 2048
    + temperature: 0.3
    cost_per_1k_tokens: 0.0
    - alias: "slm"
    - model: "prism-ml/bonsai-27b"
    + model: "qwen2.5-coder-7b-instruct"
    capabilities: ["code", "reasoning"]
    - context_window: 32768
    - max_output_tokens: 8192
    - temperature: 0.7
    + context_window: 4096
    + max_output_tokens: 2048
    + temperature: 0.3
    cost_per_1k_tokens: 0.0

```

```

- alias: "smart"
- model: "prism-ml/bonsai-27b"
+ model: "qwen2.5-coder-7b-instruct"
capabilities: ["code", "reasoning"]
- context_window: 32768
- max_output_tokens: 8192
- temperature: 0.7
+ context_window: 4096
+ max_output_tokens: 2048
+ temperature: 0.3
cost_per_1k_tokens: 0.0
- alias: "judge"
- model: "prism-ml/bonsai-27b"
+ model: "qwen2.5-coder-7b-instruct"
capabilities: ["reasoning"]
- context_window: 32768
- max_output_tokens: 4096
- temperature: 0.2
+ context_window: 4096
+ max_output_tokens: 1024
+ temperature: 0.1
cost_per_1k_tokens: 0.0
- alias: "slm-judge"
- model: "prism-ml/bonsai-27b"
+ model: "qwen2.5-coder-7b-instruct"
capabilities: ["reasoning"]
- context_window: 32768
- max_output_tokens: 4096
- temperature: 0.2
+ context_window: 4096
+ max_output_tokens: 1024
+ temperature: 0.1
cost_per_1k_tokens: 0.0
# Cloud Gemini — disabled; no API key required for infrastructure bring-up.
@@ -93,8 +94,9 @@ spec:
default_provider: "lmstudio"
max_retries: 2
retry_delay_ms: 5000
- # Wall clock for a single LLM generate (seconds).
- llm_overall_timeout_secs: 300
+ # Wall clock for a single LLM generate (seconds). Qwen 7B Q4_K_M
+ # finishes well under this; raise if switching back to a thinking 27B.
+ llm_overall_timeout_secs: 180
runtime:
container_socket_path: "/run/podman/podman.sock"

```

podman/pods/core/pod-core.yaml

```

diff --git a/podman/pods/core/pod-core.yaml b/podman/pods/core/pod-core.yaml
index 8f5df44..9630e21 100644
--- a/podman/pods/core/pod-core.yaml
+++ b/podman/pods/core/pod-core.yaml
@@ -8,6 +8,13 @@ metadata:
app: aegis
context: core
spec:
+ # netavark does not route pasta's 169.254.1.2. deploy.sh patches /etc/hosts
+ # to the host LAN IP; this alias is a second source for the same mapping.
+ hostAliases:
+ - ip: "${AEGIS_HOST_LAN_IP}"
+ hostnames:
+ - host.containers.internal

```

```
+ - host.docker.internal
containers:
# --- AEGIS Runtime ---
- name: aegis-runtime
```

scripts/apply-slow-slm-overlays.sh

```
diff --git a/scripts/apply-slow-slm-overlays.sh b/scripts/apply-slow-slm-overlays.sh
new file mode 100755
index 00000000..fccac9a
--- /dev/null
+++ b/scripts/apply-slow-slm-overlays.sh
@@ -0,0 +1,76 @@
#!/usr/bin/env bash
## Deploy manifests/slow-slm overlays (latest version) so local SLM timeouts
## stay above stock v1.0.0 builtins that core re-registers on startup.
+set -euo pipefail
+
+SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
+ROOT_DIR="$(dirname "$SCRIPT_DIR")"
+OVERLAY_DIR="$ROOT_DIR/manifests/slow-slm"
+RUNTIME_URL="${AEGIS_RUNTIME_URL:-http://127.0.0.1:8088}"
+KEYCLOAK_URL="${KEYCLOAK_URL:-http://127.0.0.1:8180}"
+
+if [[ -f "$ROOT_DIR/.env" ]]; then
+ set -a
+ # shellcheck source=/dev/null
+ source "$ROOT_DIR/.env"
+ set +a
+fi
+
+echo "==> Waiting for runtime at $RUNTIME_URL/health ..."
+for i in $(seq 1 60); do
+ if curl -sf --max-time 3 "$RUNTIME_URL/health" >/dev/null 2>&1; then
+ echo "runtime is up"
+ break
+ fi
+ if [[ "$i" -eq 60 ]]; then
+ echo "ERROR: runtime not healthy after 60s at $RUNTIME_URL/health"
+ exit 1
+ fi
+ sleep 2
+done
+
+echo "==> Requesting Keycloak token ..."
+CLIENT_SECRET="${KEYCLOAK_CLIENT_SECRET:-placeholder}"
+TOKEN=$(curl -sS -X POST "$KEYCLOAK_URL/realms/aegis-system/protocol/openid-connect/token" \
+ -H "Content-Type: application/x-www-form-urlencoded" \
+ -d "client_id=aegis-runtime" \
+ --data-urlencode "client_secret=${CLIENT_SECRET}" \
+ -d "grant_type=client_credentials" \
+ | python3 -c "import sys,json; print(json.load(sys.stdin)['access_token'])")
+
+if [[ -z "$TOKEN" || "$TOKEN" == "null" ]]; then
+ echo "ERROR: could not obtain access token from $KEYCLOAK_URL"
+ exit 1
+fi
+
+export AEGIS_KEY="$TOKEN"
+export AEGIS_API_TOKEN="$TOKEN"
+export AEGIS_HOST=127.0.0.1
+export AEGIS_PORT=8088
+
```

```

+AEGIS_BIN="${AEGIS_BIN:-$HOME/.local/bin/aegis}"
+if [[ ! -x "$AEGIS_BIN" ]]; then
+ AEGIS_BIN="$(command -v aegis || true)"
+fi
+if [[ -z "$AEGIS_BIN" ]]; then
+ echo "ERROR: aegis CLI not found"
+ exit 1
+fi
+
+echo "==> Deploying slow-slm agent overlays ..."
+shopt -s nullglob
+agents=("$OVERLAY_DIR"/aegis-*-agent.yaml)
+if [[ ${#agents[@]} -eq 0 ]]; then
+ echo "ERROR: no agent overlays in $OVERLAY_DIR"
+ exit 1
+fi
+for f in "${agents[@]"; do
+ echo " agent $(basename "$f")"
+ "$AEGIS_BIN" --host 127.0.0.1 --port 8088 agent deploy --force "$f"
+done
+
+echo "==> Deploying slow-slm workflow overlay ..."
+"$AEGIS_BIN" --host 127.0.0.1 --port 8088 workflow deploy --force --scope global \
+ "$OVERLAY_DIR/builtin-intent-to-execution.yaml"
+
+echo "==> Slow-SLM overlays applied (latest = overlay version in $OVERLAY_DIR)"

```

scripts/bootstrap-keycloak.sh

```

diff --git a/scripts/bootstrap-keycloak.sh b/scripts/bootstrap-keycloak.sh
index ae58cc2..c3d96cc 100755
--- a/scripts/bootstrap-keycloak.sh
+++ b/scripts/bootstrap-keycloak.sh
@@ -124,8 +124,9 @@ echo "Client aegis-runtime created or exists"
# Create custom client scopes needed by runtime + temporal worker
API_SCOPES=(
"agent:list" "agent:read" "agent:execute" "agent:deploy" "agent:update"
- "execution:read" "execution:stream"
+ "execution:read" "execution:stream" "execution:list"
"workflow:read" "workflow:list" "workflow:run" "workflow:execute" "workflow:deploy"
+ "workflow:logs" "workflow:cancel" "workflow:status"
"volume:read" "volume:write"
)
for SCOPE in "${API_SCOPES[@]"; do

```

scripts/deploy.sh

```

diff --git a/scripts/deploy.sh b/scripts/deploy.sh
index 01bcceb..4819370 100755
--- a/scripts/deploy.sh
+++ b/scripts/deploy.sh
@@ -8,6 +8,14 @@ PODS_DIR="$ROOT_DIR/podman/pods"
# shellcheck source=lib/systemd-user.sh
source "$SCRIPT_DIR/lib/systemd-user.sh"
+# shellcheck source=lib/host-lan-ip.sh
+source "$SCRIPT_DIR/lib/host-lan-ip.sh"
+
+AEGIS_HOST_LAN_IP="${AEGIS_HOST_LAN_IP:-$(detect_host_lan_ip)}"
+export AEGIS_HOST_LAN_IP
+if [[ -n "$AEGIS_HOST_LAN_IP" ]]; then
+ echo "Host LAN IP (host.containers.internal): $AEGIS_HOST_LAN_IP"
+fi

```

```
# Load environment
if [[ -f "$ROOT_DIR/.env" ]]; then
@@ -87,5 +95,17 @@ for pod in $PODS; do
fi
done

+echo ""
+if [[ " $PODS " == *" core "*" ]]; then
+ echo " -> Patching host.containers.internal inside core..."
+ bash "$SCRIPT_DIR/patch-host-gateway.sh" || echo "WARNING: host gateway patch failed"
+ echo " -> Applying slow-SLM overlays (local LM Studio timeouts)..."
+ if bash "$SCRIPT_DIR/apply-slow-slm-overlays.sh"; then
+ echo " -> Slow-SLM overlays applied"
+ else
+ echo "WARNING: slow-SLM overlays were not applied. Run: make overlays"
+ fi
+fi
+
echo ""
echo "Deployment complete. Run 'make status' to verify."
```

scripts/lib/host-lan-ip.sh

```
diff --git a/scripts/lib/host-lan-ip.sh b/scripts/lib/host-lan-ip.sh
new file mode 100644
index 0000000..9cbfff3
--- /dev/null
+++ b/scripts/lib/host-lan-ip.sh
@@ -0,0 +1,10 @@
+# Detect the host IPv4 that containers can hairpin to on a netavark bridge.
+# pasta's host.containers.internal (169.254.1.2) is unreachable from aegis-network.
+detect_host_lan_ip() {
+ ip -4 route get 1.1.1.1 2>/dev/null \
+ | awk '{for (i = 1; i <= NF; i++) if ($i == "src") { print $(i + 1); exit } }'
+}
+
+if [[ "${BASH_SOURCE[0]}" == "${0}" ]]; then
+ detect_host_lan_ip
+fi
```

scripts/patch-host-gateway.sh

```
diff --git a/scripts/patch-host-gateway.sh b/scripts/patch-host-gateway.sh
new file mode 100755
index 0000000..137a73a
--- /dev/null
+++ b/scripts/patch-host-gateway.sh
@@ -0,0 +1,45 @@
#!/usr/bin/env bash
+# Point host.containers.internal at the host LAN IPv4 inside aegis-core.
+# On Kali + netavark, Podman injects 169.254.1.2 which times out from the
+# bridge network. LM Studio and the FUSE daemon bind 0.0.0.0 on the host.
+set -euo pipefail
+
+SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
+# shellcheck source=lib/host-lan-ip.sh
+source "$SCRIPT_DIR/lib/host-lan-ip.sh"
+
+CONTAINER="${1:-aegis-core-aegis-runtime}"
+HOST_IP="${AEGIS_HOST_LAN_IP:-$(detect_host_lan_ip)}"
+
+if [[ -z "$HOST_IP" ]]; then
+ echo "ERROR: could not detect host LAN IP (set AEGIS_HOST_LAN_IP)"
```

```

+ exit 1
+fi
+
+if ! podman container exists "$CONTAINER" 2>/dev/null; then
+ echo "WARN: $CONTAINER not running – skip host.containers.internal patch"
+ exit 0
+fi
+
+echo " -> Mapping host.containers.internal -> $HOST_IP in $CONTAINER"
+podman exec -u 0 "$CONTAINER" sh -c "
+HOSTIP='$HOST_IP'
+awk -v ip=\"\$HOSTIP\" '
+ BEGIN { done = 0 }
+ /host.containers.internal/ {
+ print ip \" host.containers.internal host.docker.internal\"
+ done = 1
+ next
+ }
+ { print }
+ END { if (!done) print ip \" host.containers.internal host.docker.internal\" }
+' /etc/hosts > /tmp/hosts.new
+cat /tmp/hosts.new > /etc/hosts
+" >/dev/null
+
+if podman exec "$CONTAINER" curl -sf --max-time 5 "http://host.containers.internal:1234/v1/models"
>/dev/null 2>&1; then
+ echo " -> host.containers.internal:1234 reachable (LM Studio)"
+else
+ echo "WARN: host.containers.internal:1234 not reachable from $CONTAINER"
+ echo " Bind LM Studio to 0.0.0.0 and confirm the host firewall allows $HOST_IP:1234"
+fi

```

With respect and thanks — we will take a Qwen-backed fib run next. This document covers the Bonsai era plus the cutover. Overlay YAMLS and the unified diff above are yours if you want them upstream. AEGIS is a remarkable piece of work.